



Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 2

Software Development Methodologies, Version Control,
Software Project Brainstorming & Team Formations



How Software Is Built in Industry Today

- Product teams
 - Cross-functional (engineering, product, design, QA)
 - Long-lived teams owning a product, not short projects
 - Teams responsible for success after release
- Continuous delivery
 - Software is always in a “potentially releasable” state
 - Small, frequent changes instead of large releases
 - Automation enables speed and safety

Why might companies prefer frequent, smaller releases over fewer, larger ones?



What Is a Software Methodology?

- Structured approach to building software
- Organizes work into phases or activities
- Defines roles and responsibilities
- Guides how change is handled



Why Software Methodologies Exist

- Uncertainty about requirements and outcomes
- Risk of failure and wasted effort
- Coordination across people and time
- Need for predictability and accountability



Two Broad Approaches

- Plan-driven: Waterfall
- Adaptive: Agile



Origins of Waterfall

- Rooted in traditional engineering disciplines
- Influenced by construction and manufacturing
- Emphasized predictability and control
- Formalized in early software projects



Waterfall Software Methodology

- Waterfall overview
 - Linear sequence of phases
 - Heavy upfront requirements and design
 - Testing late in the process
- Strengths
 - Predictable milestones
 - Works well when requirements are stable
 - Clear documentation and contracts
- Weaknesses
 - Late feedback
 - High cost of change
 - Risk of building the wrong system



Waterfall Lifecycle

- Requirements
- Design
- Implementation
- Testing
- Deployment
- Maintenance



Requirements Analysis

- Define what the system must do
- Gather functional and non-functional requirements
- Produce formal requirements documentation
- Stakeholder review and sign-off



System & Software Design

- Translate requirements into technical solutions
- Define architecture and components
- Choose technologies and data models
- Produce detailed design specifications



Implementation

- Code is written according to design
- Developers follow specifications closely
- Limited deviation from approved design
- Progress measured by feature completion



Testing and Verification

- Verify system meets requirements
- Testing often occurs late
- Bugs and defects are documented
- Fixes may require revisiting earlier phases



Deployment

- System released to users
- Installation and configuration
- User training and documentation
- Transition to operational use



Maintenance

- Bug fixes and minor updates
- Changes handled through formal process
- New features often trigger new projects
- System eventually retired or replaced



Strengths of the Waterfall Methodology

- Clear structure and documentation
- Predictable timelines and deliverables
- Works well in stable, regulated domains
- Easier to manage formally



Weaknesses of the Waterfall Methodology

- Late user feedback
- Poor adaptability to change
- High cost of late corrections
- Risk of building the wrong system



Why Agile Emerged

- Software requirements change frequently
- Users don't know what they want until they see it
- Late feedback creates high risk
- Planning cannot eliminate uncertainty



Core Idea of Agile

- Software is discovered, not fully specified
- Learning happens through working software
- Feedback reduces risk more than prediction
- Progress is incremental



Agile Manifesto

- Written in 2001 by experienced developers frustrated with process-heavy failures
- Response to heavyweight, document-driven processes
- Focused on values, not rules or tools



Agile Manifesto

Core Values

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

<https://agilemanifesto.org/principles.html>



What Agile is Not

- Not no planning
- Not no documentation
- Not chaos or lack of discipline
- Not “just go fast”



Iterative & Incremental Development

- Work delivered in small increments
- Short cycles (often weeks)
- Each iteration produces usable software
- Feedback informs the next cycle



User Stories & Backlogs

- Requirements expressed as user needs
- Focus on value, not implementation
- Details refined over time
- Prioritized continuously



Strengths of Agile

- Early and frequent feedback
- Adaptable to change
- Reduces risk of building the wrong thing
- Aligns with modern software environments



Risks & Challenges of Agile

- Requires strong communication
- Demands discipline and ownership
- Harder to manage without trust
- Poorly executed Agile fails loudly



Scrum Overview

- Roles
 - Product Owner: Values and Priorities
 - Scrum Master: Process and Impediments
 - Development Team: Builds the Product
- Ceremonies
 - Sprint Planning
 - Daily Standups
 - Sprint Review
 - Retrospective
- Artifacts
 - Product Backlog
 - Sprint Backlog
 - Increment



Agile in Practice

- What teams really do vs. Textbook Scrum
 - Hybrid processes
 - Fewer ceremonies, more communication
 - Adjust practices over time
- Alternatives to Scrum
 - Kanban: flow-based, no fixed sprints
 - Extreme Programming: engineering discipline (TDD, pair programming)
 - Lean: minimize waste, optimize the whole system



Agile vs Waterfall (Revisited)

- Waterfall manages uncertainty by planning
- Agile manages uncertainty by feedback
- Most real teams blend approaches



Why Version Control Matters

- Enables collaboration without overwriting work
- Allows multiple people to work on the same codebase safely
- Maintains a complete history of changes
- Records who changed what, when, why
- Makes experimentation low-risk
- Supports rollback and recovery from mistakes

Version control is essential infrastructure for team-based software development.



Git: Core Concepts

- Git is Distributed version control
- Repository
 - Shared project workspace
 - Contains source code, configuration, documentation – anything related to project that you want to track
 - Exists locally and remotely (e.g. GitHub)
- Commit
 - Snapshot of a logical change
 - Includes author, timestamp, and message
 - Serves as technical documentation



Core Git Concepts (continued)

- Branch
 - Independent line of development
 - Enables parallel work
 - Keeps unfinished work isolated
 - Can be permanent or temporary
- Merge
 - Combines changes from different branches
 - Integrates completed work into shared codebase
 - Conflicts are expected and manageable



Git Workflow Demo

- One Repo (Remote repo on GitHub)
- Two users/contributors
- Handling merge conflicts
- Branching and pull requests



Typical Team Git Workflow

- Main branch represents stable, working software
- New work happens on feature branches
- Developers commit small, focused changes
- Pull requests propose changes for integration
- Automatic checks run before merging
- Code review provides feedback and quality control, often a requirement for approving a pull request

Git workflows create structured feedback loops



Common Git Mistakes

- Working directly on the main branch
- Making large, unfocused commits
- Skipping code reviews
- Writing poor or vague commit messages
- Treating Git as a backup tool instead of a collaboration tool

Most Git problems result from poor process, not Git itself



How Git Supports Agile Software Development

- Enables Incremental Development
- Supports frequent integration and delivery
- Allows multiple features to be developed in parallel
- Provides traceability from work items to code changes
- Makes feedback and iteration visible

Agile practices depend on version control to scale



Further Git Resources and Exercises

- Git Reference Book
<https://git-scm.com/book/en/v2>
- Using Git with GitHub by Bill Mongan
<https://www.billmongan.com/posts/2020/02/github/>
- Git Workshop from Bryn Mawr College
<https://github.com/BrynMawr-CS223-S22/git-workshop>
- Learn Git Branching
<https://learngitbranching.js.org/>
- Git-workflow vs feature branching
<https://gist.github.com/blackfalcon/8428401>



Assignment: Semester Project Proposal

<https://rrostock1.github.io/Ursinus-CS375/Project/Proposal>

Due Feb 12



Assignment: Git

<https://rrostock1.github.io/Ursinus-CS375/Assignments/git>

Due Feb 19